

Course: Microprocessors	Section: 3-6
Name/s: Carl Errol Cabali	Date Performed: June 1, 2026
	Date Submitted: June 10, 2026
	Instructor: Engr. Rufo I. Marasigan Jr.
Laboratory Activity No. 5	
Working on Motors	
1. Objective(s):	
This laboratory activity introduces students to the basic concepts, operation, and interfacing of electric motors using the Arduino platform. The focus is on controlling servo and DC motors through various methods, such as relay modules and motor driver circuits. It also aims to enhance students' understanding of digital input and output operations, pulse-width modulation (PWM), and motor control logic. Furthermore, the activity provides hands-on experience in developing Arduino programs that utilize conditional statements, loops, and input devices (e.g., potentiometers, tact switches, and joysticks) to implement motor-based automation and directional control systems.	
2. Intended Learning Outcomes (ILOs):	
At the end of this activity, students shall be able to: ● Explain the basic principles and operation of electric motors, servo motors, relays, and motor driver circuits. ● Interface and control a servo motor using Arduino and PWM-based signal control. ● Interface and control a DC motor using relay modules and L293D motor driver circuits. ● Apply digital input/output operations, loops, and conditional statements in Arduino motor control applications. ● Manipulate and modify Arduino C/C++ programs to implement required motor movement and control sequences. ● Design simple motor automation systems using switches, potentiometers, and joystick modules. ● Troubleshoot basic hardware and software issues encountered during motor interfacing and control activities.	
3. Materials	
The following materials and components available in Tinkercad Circuits will be used in this laboratory activity: ● Arduino Uno R3 ● Breadboard ● Jumper wires ● Servo Motor ● DC Motor ● Relay	

- L293D Motor Driver IC
- Potentiometer
- Tact Switches (Push Buttons)
- Joystick Module
- LEDs (optional for status indication)
- Resistors
- External power source component available in Tinkercad (if needed)
- Computer or laptop with internet connection
- Tinkercad Circuits simulation environment

4. Discussion

Electric Motor

An electric motor converts electrical energy into mechanical energy. It operates by generating rotary motion when current flows within a magnetic field. Motors are broadly classified by their power source (AC or DC) and can be further categorized by their rotational principles, as conceptually illustrated in Figure 5.1.

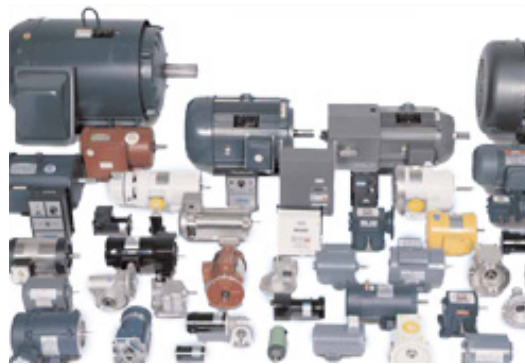


Figure 5.1 - Electric Motors

How a Motor Works (Source: rohm.com)

Motor Rotation Using a Magnet/Magnetic Force

Principle of a permanent magnet with a rotational axis (Figure 5.2):

- ① When the outer magnets rotate (referred to as a rotating magnetic field),
- ② The N and S poles attract and repel each other,
- ③ Causing the magnet with the rotational axis (center) to turn.

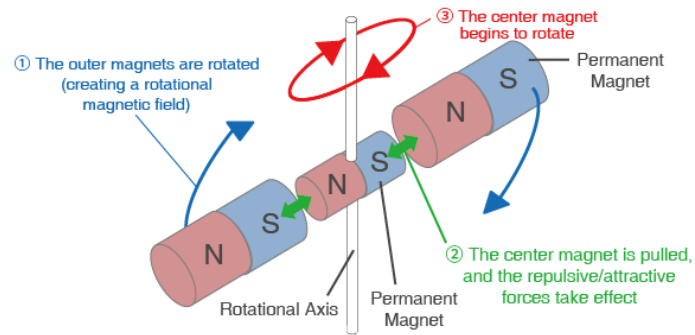


Figure 5.2 – Electric Motor

Alternatively, supplying current to a conductor generates a magnetic field around it, creating a rotating magnetic force. This produces the same effect as rotating a physical magnet. The magnetic lines of force are shown in Figure 5.3.

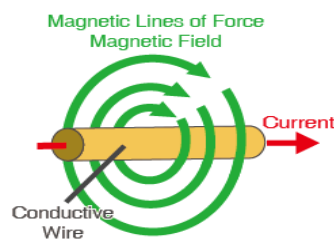


Figure 5.3 – Magnetic Line f Force in Electric Motor

Winding a conductive wire into a coil combines the magnetic force, generating a large magnetic flux with defined North and South poles (Figure 5.4).

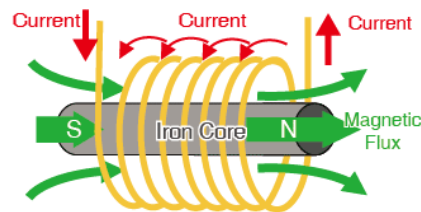


Figure 5.4 – Electric motor coil wound with a conductive wire

Actual Motor Operation

This section illustrates a method for rotating a motor by generating a rotating magnetic field from a 3-phase AC source and conductive coils (see Figure 5.5).

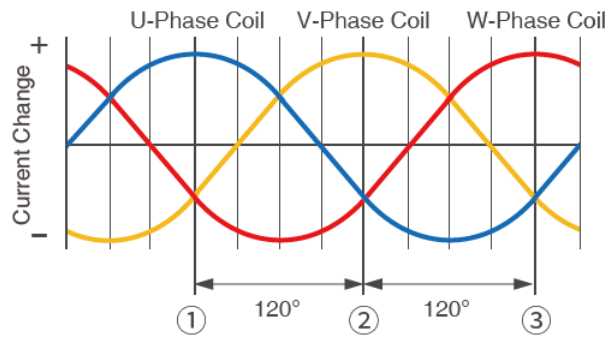


Figure 5.5 - Graph of rotating Electric Motor

- The synthetic magnetic field in ① above is shown in ① below
- The synthetic magnetic field in ② above is shown in ② below
- The synthetic magnetic field in ③ above is shown in ③ below

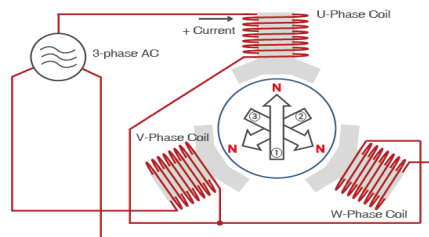


Figure 5.6 - Electric Motor Internal Connection

Figure 5.6 shows the internal connection and phase arrangement.

As mentioned, the three-phase coils (U, V, and W) are wrapped around an iron core and positioned 120° apart. Higher voltage generates a North pole (N) while lower voltage generates a South pole (S). Since each phase varies sinusoidally, the poles generated by each coil—and the resulting magnetic field—change continuously.

If we look at just the North pole phase, it switches in order from the U coil $\rightarrow$ V coil $\rightarrow$ W coil $\rightarrow$ U coil, enabling rotation.

DC Motor

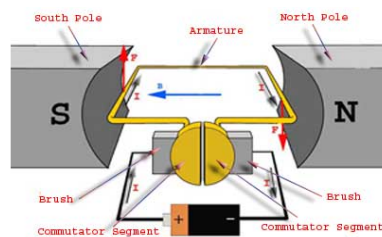


Figure 5.7 – Structure of DC motor

A DC (direct current) motor works on the principle that when a current-carrying conductor is placed in a magnetic field, it experiences a torque and tends to move; this is known as motoring action. Reversing the current direction reverses the rotation. The working principle of a DC motor is established through the interaction of magnetic and electric fields to produce mechanical force (Figure 5.7).

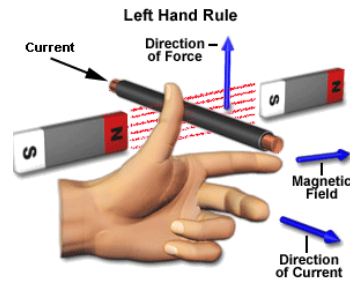


Figure 5.8 – Fleming's Left Hand Rule

The direction of rotation is determined by Fleming's Left-Hand Rule (Figure 5.8): if the index finger, middle finger, and thumb of the left hand are extended mutually perpendicular to each other, and the index finger represents the magnetic field while the middle finger represents the current, then the thumb indicates the direction of the force experienced by the motor shaft. (Source: electrical4u.com)

Servo Motor

In certain applications, a motor is required to rotate to a specific angle rather than continuously. Servo motors are designed for this purpose, using a special arrangement to rotate to a certain angle based on an electrical input signal. A typical servo is a DC motor controlled for specific angular rotation via a servomechanism (a closed-loop feedback system). Today, servo systems have extensive industrial applications (Figure 5.9).

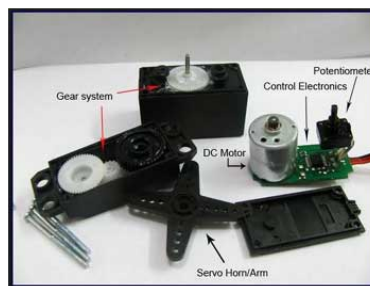


Figure 5.9 – Servo Motors

Electric Relays

A relay is an electromagnetic switch operated by a relatively small electric current that can toggle a much larger current. It contains an electromagnet (a coil of wire that becomes temporary magnet when electricity flows through it). Think of a relay as an electric lever: switching it on with a tiny current triggers another appliance using a much larger current. This is useful because many sensors are sensitive and produce only small currents, yet they often need to drive larger apparatuses. Relays bridge this gap, acting as either switches or amplifiers. (Source: explainthatstuff.com)

How Does Relay Works?

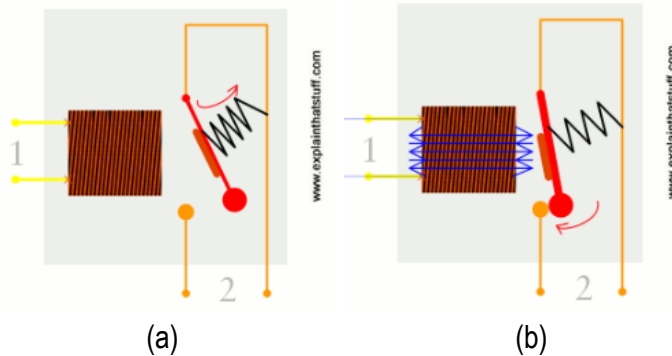


Figure 5.10 – Relay on Actions. (a) Unactive connection; (b) Active connection

This is an example of a "normally open" (NO) relay: the contacts in the second circuit are not connected by default, and switch on only when a current flows through the magnet. Other relays are "normally closed" (NC; the contacts are connected so a current flows through them by default) and switch off only when the magnet is activated, pulling or pushing the contacts apart. Normally open relays are the most common.

5. Procedures

Part I: Interfacing Servo Motor to Arduino

A servo motor is a type of motor that allows precise control angular position. Unlike a regular DC motor that continuously rotates, a servo motor can rotate to a specific angle based on the signal sent by the Arduino. In this activity, a potentiometer will be used as an input device to control the angle of the servo motor.

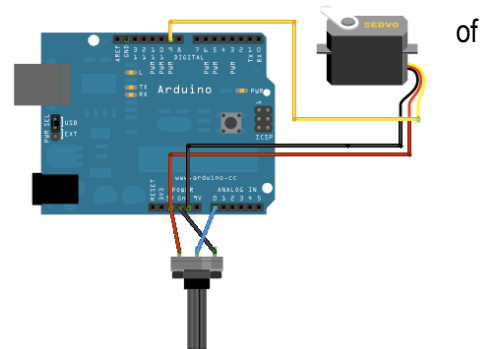


Figure 5.11 – Servo Motor Connection

The servo motor has three terminals:

- VCC (Power) – usually the red wire, connected to the Arduino 5V pin
- GND (Ground) – usually the black or brown wire, connected to GND
- Signal – usually the yellow or orange wire, connected to a PWM-capable pin of the Arduino

The potentiometer will serve as the manual controller of the servo motor angle. Rotating the potentiometer changes the analog voltage input read by the Arduino, which is then converted into the corresponding servo rotation angle.

Circuit Connections

The table below outlines the connections for the servo motor and potentiometer used in this activity:

Table 5.1 – Servo Motor Connections

Servo Motor Pin	Arduino Pin
+5v Pin (Red)	+5V
Grnd (Black or Brown)	GND
Input (Yellow)	Pin9

Table 5.2 – Potentiometer Connections

Potentiometer Pin	Arduino Pin
Left Pin	+5V
Right Pin	GND
Middle Pin	A0

Procedure

1. Open **Tinkercad Circuits** and create a new circuit project.
2. Add the following components to the workspace:
 - a. Arduino Uno R3
 - b. Servo Motor
 - c. Potentiometer
3. Connect the servo motor:
 - a. Connect the **red wire** to the **5V** pin of the Arduino.
 - b. Connect the **black/brown wire** to the **GND** pin.
 - c. Connect the **yellow/orange signal wire** to **digital pin 9**.
4. Connect the potentiometer:
 - a. Connect one outer pin to **5V**.
 - b. Connect the other outer pin to **GND**.
 - c. Connect the middle pin to **analog pin A0**.
5. Verify all connections before running the simulation.
6. Open the **Code Editor** and upload the provided Arduino program.

```
// Part I: Interfacing Servo Motor with Arduino
// Control servo angle using potentiometer

#include <Servo.h>

Servo myServo;

int potPin = A0;
int potValue = 0;
int servoAngle = 0;

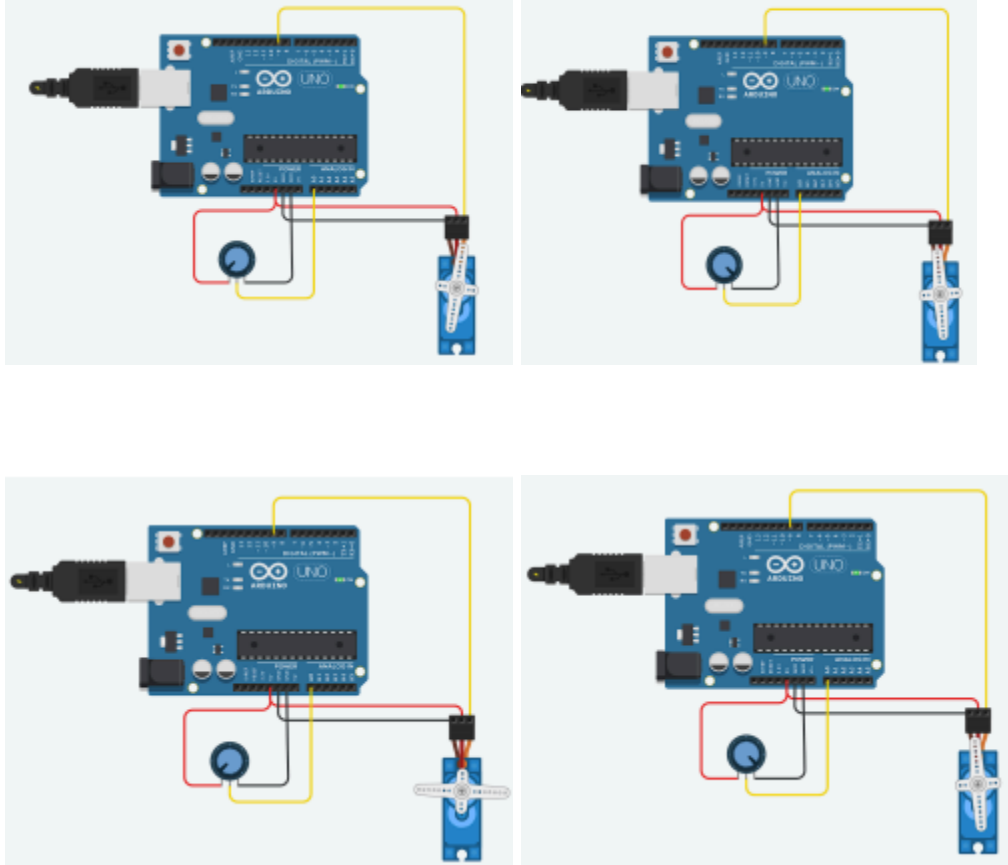
void setup() {

  // Attach servo signal pin to Arduino pin 9
  myServo.attach(9);
}

void loop() {

  // Read potentiometer value
  potValue = analogRead(potPin);
```

7. Start the simulation.
8. Rotate the potentiometer slowly and observe the movement of the servo motor.
9. Record your observations regarding the relationship between the potentiometer position and the servo motor angle.
10. Modify the value 180 in the `map()` function to 90.
11. Run the simulation again and observe the changes in the servo motor movement.
12. Answer the following guide question:



Guide Question

1. What happened to the movement range of the servo motor after changing the value from 180 to 90 in the `map()` function?

I connected a servo motor and a potentiometer to an Arduino. The program read the potentiometer's input value and adjusted the servo motor's position accordingly. When I modified the `map()` function from a range of 0–180 to 0–90, the servo's rotation was limited to 90 degrees instead of the full 180 degrees. This demonstrated that the `map()` function determines the servo motor's range of motion.

Part 2: Interfacing DC motor on Arduino using Relay

A relay is an electrically controlled switch that allows a low-power circuit, such as an Arduino output pin, to control a higher-power device like a DC motor. In this activity, the relay will be used to turn the DC motor ON and OFF using a digital signal from the Arduino.

When the Arduino sends a HIGH signal to the relay module, the relay activates and completes the motor circuit, causing the motor to rotate. When the signal becomes LOW, the relay deactivates and disconnects the motor circuit, stopping the motor.

Relay Connections

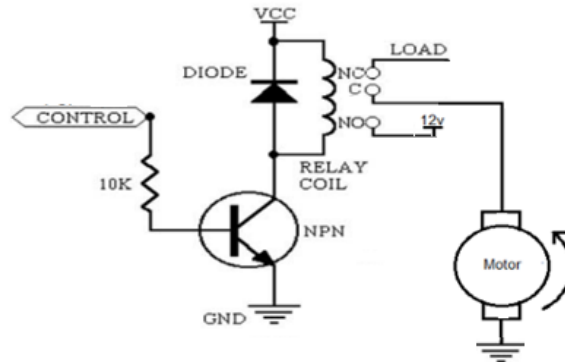


Figure 5.12 DC motor Connected to Relay

The tables below outline the connections for the relay and DC motor used in this activity:

Table 5.3 – Relay Connections

Relay Pin	Arduino Pin
VCC	+5V
GND)	GND
IN (Signal)	Pin9

Table 5.4 – DC Motor Connection (Relay)

Component	Connection
DC Motor Terminal 1	Relay NO Terminal
DC Motor Terminal 2	Ground / External Supply
Relay COM Terminal	Positive Supply

Procedure

1. Open **Tinkercad Circuits** and create a new circuit project.
2. Add the following components:
 - Arduino Uno R3
 - Relay
 - DC Motor
 - Battery or Power Supply
 - Breadboard (optional)
3. Connect the relay module to the Arduino:

- Connect the **VCC** pin of the relay to **5V**.
 - Connect the **GND** pin to **GND**.
 - Connect the **IN** pin to **digital pin 9**.
4. Connect the DC motor and external power supply through the relay:
 - Connect the positive terminal of the supply to the **COM** terminal of the relay.
 - Connect the **NO** terminal of the relay to one terminal of the DC motor.
 - Connect the other terminal of the DC motor to the negative terminal of the supply.
 5. Double-check all wiring connections before starting the simulation.
 6. Open the **Code Editor** and upload the provided Arduino program.

```
// Part II: Interfacing DC Motor Using Relay
// The relay will turn the DC motor ON and OFF
repeatedly

const int relayPin = 9;
void setup() {

    // Set relay pin as OUTPUT
    pinMode(relayPin, OUTPUT);
}

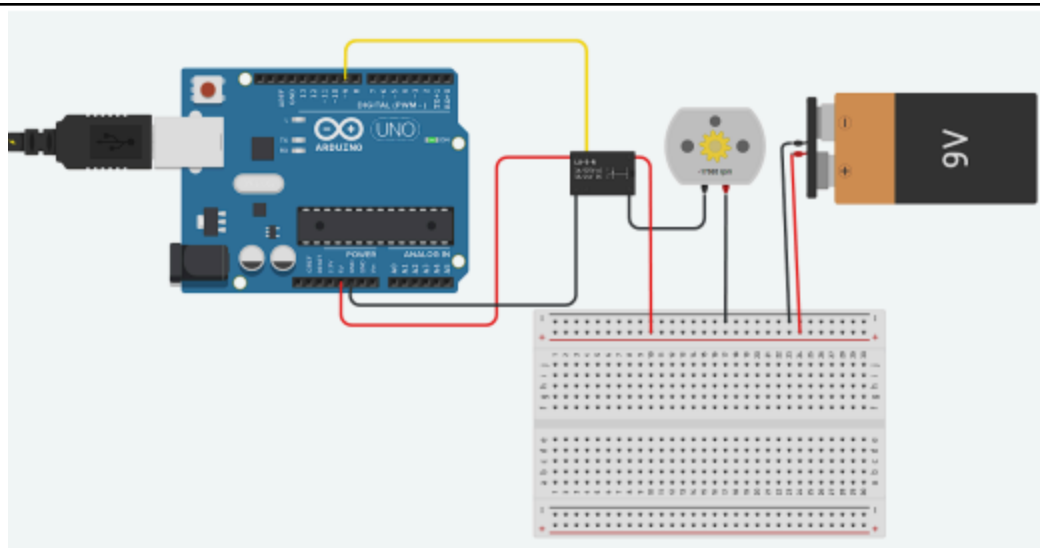
void loop() {
    // Turn relay ON
    digitalWrite(relayPin, HIGH);

    // DC motor rotates
    delay(3000);

    // Turn relay OFF
    digitalWrite(relayPin, LOW);

    // DC motor stops
    delay(3000);
}
```

7. Start the simulation.
8. Observe the behavior of the relay and the DC motor.
9. Listen for the clicking sound or observe the relay switching state while the motor turns ON and OFF.
10. Record your observations regarding the operation of the relay and the DC motor.



Guide Questions

2. What happens to the DC motor when the relay receives a HIGH signal from the Arduino?

When the relay received a HIGH signal from the Arduino, it activated and closed its contacts, allowing electrical power to flow to the motor. As a result, the motor began to spin.

3. Why is a relay commonly used when controlling motors and other high-current devices?

A relay is commonly used because the Arduino cannot supply enough current for motors. The relay provides isolation and safely controls high-power devices without damaging the Arduino.

Part 3: Interfacing a DC Motor Using L293D Motor Driver

The L293D is a motor driver integrated circuit (IC) designed to control DC motors using low-current signals from a microcontroller such as Arduino. Unlike a relay that only switches the motor ON or OFF, the L293D allows the motor to rotate in both clockwise and counterclockwise directions.

The L293D works as an H-Bridge motor driver. By changing the logic of the input pins, the direction of current flow through the motor changes, causing the motor to rotate in different directions.

In this activity, the Arduino will control the rotation direction of the DC motor using the L293D motor driver:

Circuit Connections

Table 5.5 – L293D to Arduino Connections

L293D Pin	Function	Arduino Connection
Pin 1	Enable 1	Pin 5
Pin 2	Input 1	Pin 4

Pin 7	Input 2	Pin 3
Pin 8	Motor Supply	5V
Pin 16	Logic Supply	5V
Pin 4,5,12,13	Ground	GND

Table 5.6 – DC Motor Connections (L293D)

DC Motor Terminal	L293D Connection
Terminal 1	Pin 3
Terminal 2	Pin 6

Procedure

1. Open Tinkercad Circuits and create a new circuit.
2. Add the following components:
 - Arduino Uno R3
 - Breadboard
 - L293D Motor Driver IC
 - DC Motor
 - Jumper wires
3. Insert the L293D motor driver IC on the breadboard.
4. Connect the power pins:
 - Connect Pin 16 to 5V
 - Connect Pin 8 to 5V
 - Connect all ground pins to GND
5. Connect the control pins:
 - Connect Pin 1 (Enable) to Arduino Pin 5
 - Connect Pin 2 (Input 1) to Arduino Pin 4
 - Connect Pin 7 (Input 2) to Arduino Pin 3
6. Connect the DC motor terminals to:
 - L293D Pin 3
 - L293D Pin 6
7. Verify all wiring connections.
8. Open the Code Editor and upload the provided Arduino program.

```
// Part III: DC Motor Control using L293D

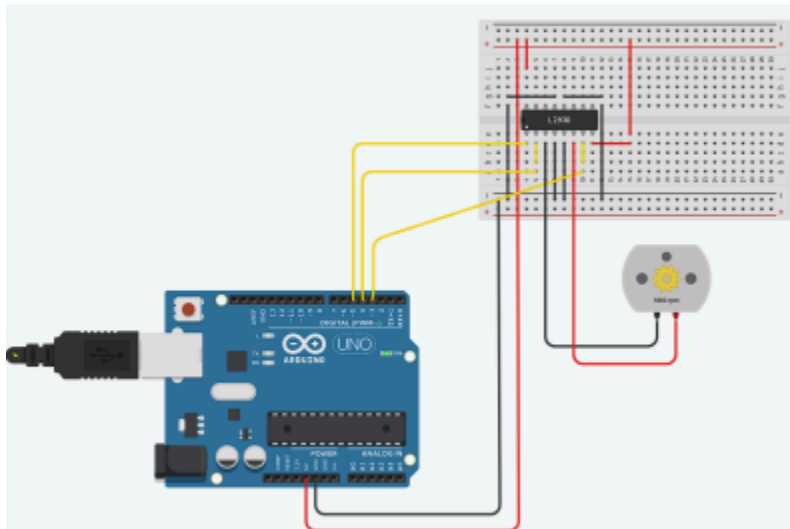
int ENA = 5;
int IN1 = 4;
int IN2 = 3;

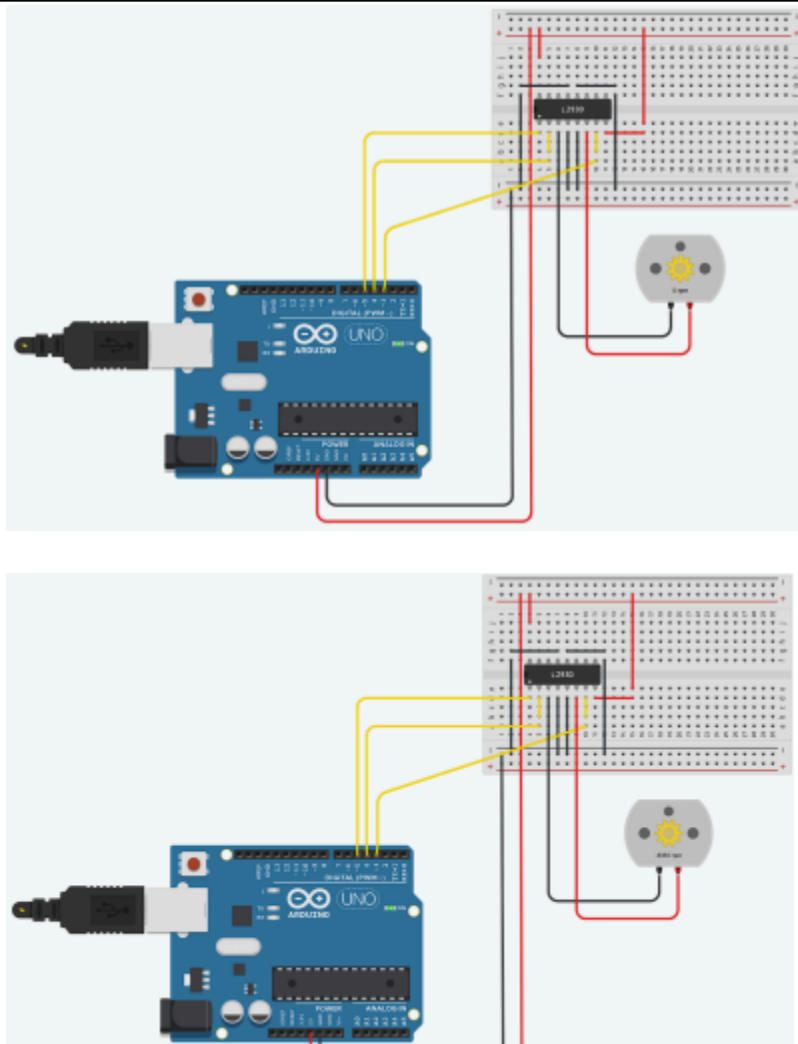
void setup() {

  pinMode(ENA, OUTPUT);
  pinMode(IN1, OUTPUT);
  pinMode(IN2, OUTPUT);
}
```

```
delay(3000);  
  
// Stop Motor  
digitalWrite(IN1, LOW);  
digitalWrite(IN2, LOW);  
delay(1000);  
}
```

9. Start the simulation.
10. Observe the direction of rotation of the DC motor.
11. Record your observations regarding clockwise and counterclockwise motor rotation.





Guide Questions

4. What happens when the logic of Input 1 and Input 2 is reversed?

When the logic states of Input 1 and Input 2 were reversed, the motor's direction of rotation also changed, causing it to rotate in the opposite direction from clockwise to counterclockwise, or vice versa.

5. What is the purpose of the Enable pin in the L293D motor driver?

The Enable pin activates or deactivates the motor driver and can also regulate the motor's speed through Pulse Width Modulation.

6. What is the advantage of using a motor driver instead of directly connecting the motor to the Arduino?

Using a motor driver is more effective than connecting a motor directly to an Arduino because it can handle higher current requirements, control the motor in both directions, and help protect the Arduino from potential damage.

6. Supplemental Activity

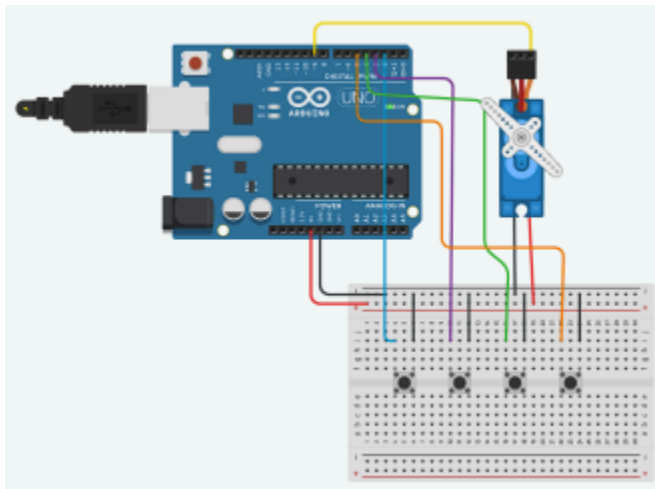
1. Create an Arduino Program that uses 4 tact switches to control the angle of the rotation of the servo motor. Table below shows the corresponding angle.

Switch No.	Servo motor Output rotation Angle
1	45
2	90
3	120
4	180

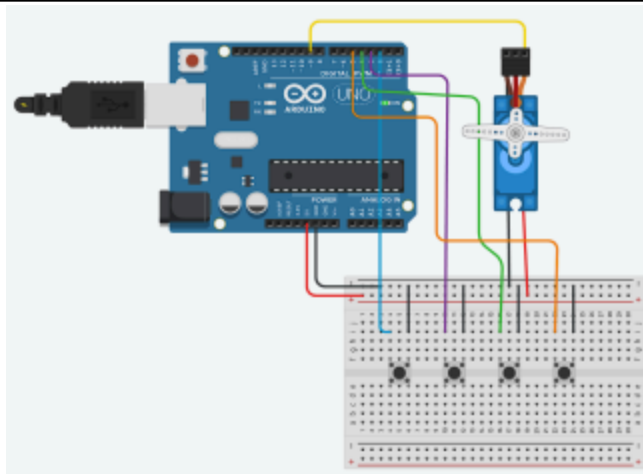
OUTPUT

```
1 #include <Servo.h>
2
3 Servo myservo;
4
5 void setup()
6 {
7   myservo.attach(9);
8
9   pinMode(2, INPUT_PULLUP);
10  pinMode(3, INPUT_PULLUP);
11  pinMode(4, INPUT_PULLUP);
12  pinMode(5, INPUT_PULLUP);
13 }
14
15 void loop()
16 {
17   if(digitalRead(2) == LOW)
18   {
19     myservo.write(45);
20   }
21
22   if(digitalRead(3) == LOW)
23   {
24     myservo.write(90);
25   }
26
27   if(digitalRead(4) == LOW)
28   {
29     myservo.write(120);
30   }
31
32   if(digitalRead(5) == LOW)
33   {
34     myservo.write(180);
35   }
36 }
```

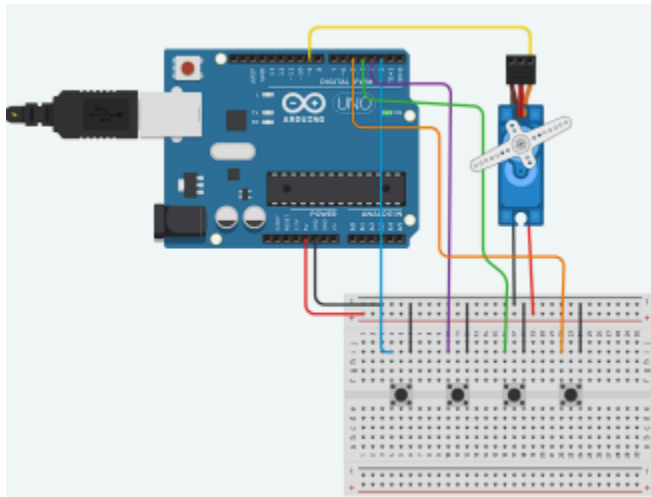
SW 1



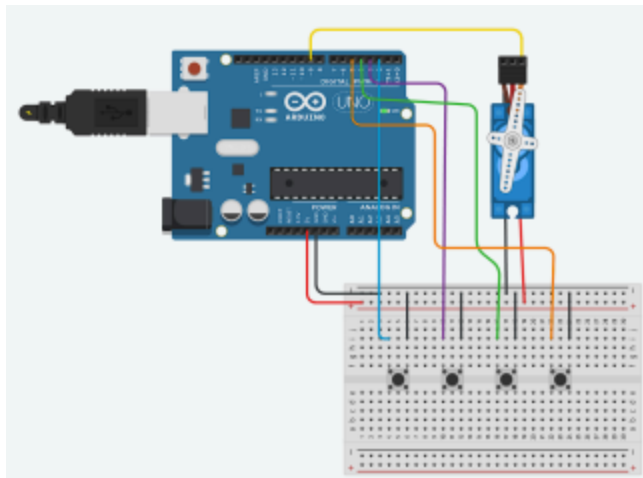
SW 2



SW 3



SW 4

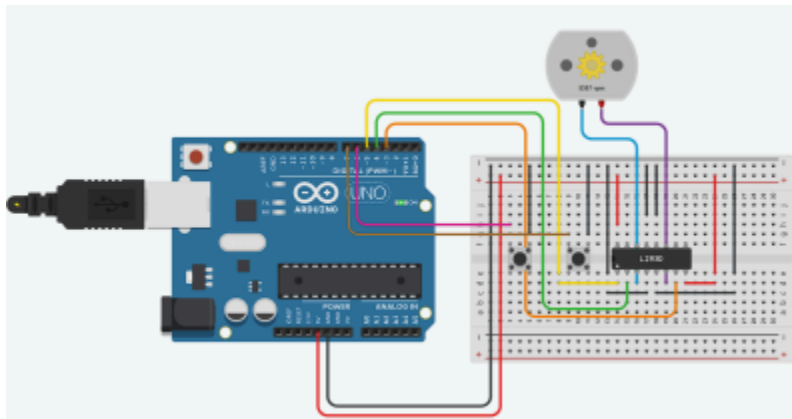


2. Create an Arduino program that controls the direction of a DC motor (clockwise or counter-clockwise) using two tact switches. You may use either a relay (referencing the provided schematic) or a motor driver.

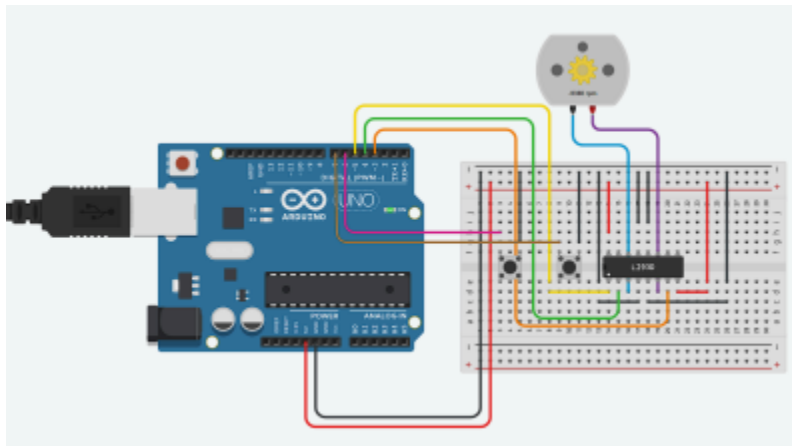
OUTPUT

```
1 void setup()
2 {
3   pinMode(5, OUTPUT); // Enable
4   pinMode(4, OUTPUT); // IN1
5   pinMode(2, OUTPUT); // IN2
6
7   pinMode(6, INPUT_PULLUP); // CW
8   pinMode(7, INPUT_PULLUP); // CCW
9
10  digitalWrite(5, HIGH); // Enable motor driver
11 }
12
13 void loop()
14 {
15   if(digitalRead(6) == LOW)
16   {
17     digitalWrite(4, HIGH);
18     digitalWrite(2, LOW);
19   }
20
21   else if(digitalRead(7) == LOW)
22   {
23     digitalWrite(4, LOW);
24     digitalWrite(2, HIGH);
25   }
26
27   else
28   {
29     digitalWrite(4, LOW);
30     digitalWrite(2, LOW);
31   }
32 }
```

SW 1



SW 2



7. Conclusion:

What conclusion can you make based on the activity?

I was able to gain a better knowledge of the motor interfacing methods frequently employed in robotics and computer engineering through this lab exercise. I applied control logic utilizing loops and conditional

statements, worked with both analog and digital I/O, and understood the significance of appropriate power management and isolation while working with actuators. The use of DC motors with drivers or relays for bidirectional control and servo motors for accurate positioning demonstrated how these parts can be integrated for a variety of automation applications. My ability to design, debug, and troubleshoot embedded systems including both hardware and software integration has also improved as a result of the exercise. In general, I feel better equipped and assured to build motor-based systems for next projects like industrial automation systems, autonomous cars, and robotic arms.

8. Assessment (Rubric for Laboratory Performance):

Intended Learning Outcomes	Unsatisfactory 1	Satisfactory 2	Exemplary 3	Score
Interface and Control Servo and DC Motors (Using Potentiometer, Relay, and L293D Driver)	Unable to properly connect or control the required motors/drivers (Servo, Relay-DC, L293D-DC).	Able to connect and control most required motors/drivers but with notable connection or configuration errors.	Successfully interfaces and controls all motors/drivers as specified in the procedures and supplemental activities.	
Apply Digital Input and Control Flow (Switches, Potentiometer, Loops, and Conditionals)	Fails to integrate input devices (potentiometer/switches) or incorrectly applies control flow structures (loops/conditionals).	Integrates input devices and uses control flow structures, but the implementation contains logical errors or does not fully satisfy the activity requirements.	Effectively integrates input devices and correctly applies loops and conditional statements to create functional, automated motor systems.	
Develop and Manipulate Arduino C/C++ Code Circuit and Program Troubleshooting	Unable to write or modify the C/C++ code to implement the required output sequences for motor movement. Fails to identify and resolve basic hardware or software	Able to write or modify the C/C++ code, but the sequence has errors or requires significant correction to achieve the required output.	Manipulates the C/C++ code efficiently and accurately to implement all required motor movement and control sequences.	

	issues during the simulation setup.	Identifies simple issues but requires guidance or significant time to resolve complex hardware or program logic errors.	Independently identifies and resolves all hardware connection issues, program logic errors, and simulation faults.	
Other comments/observation:		Total Score		
		$\text{RATING} = \frac{(\text{total score})}{9} \times 100\%$		
Evaluated by: _____ Date: _____ _____ Printed Name and Signature of Faculty Member				